

LIMPIEZA DE DATOS

Limpieza de Datos

La limpieza de datos es un proceso fundamental en el análisis de datos que consiste en identificar y corregir errores, inconsistencias e imprecisiones en un conjunto de datos. Es como limpiar una casa antes de recibir invitados: se eliminan los objetos innecesarios, se arreglan los desperfectos y se organiza todo para que sea funcional y presentable



¿Por qué es importante la limpieza de datos?

Mayor precisión en los análisis: Datos limpios garantizan resultados más confiables y precisos en cualquier análisis, ya sea estadístico, de machine learning o de business intelligence.



Toma de decisiones más informada: Al contar con datos de calidad, las empresas pueden tomar decisiones estratégicas más acertadas.



¿Por qué es importante la limpieza de datos?

Mejora de la eficiencia: Datos limpios aceleran los procesos de análisis y reducen la necesidad de volver a trabajar en los mismos datos.



Prevención de errores costosos: Identificar y corregir errores en una etapa temprana evita tomar decisiones basadas en información errónea, lo que puede tener consecuencias negativas para la empresa.



Etapas típicas de la limpieza de datos

✓ Detección de errores:

- **Valores atípicos:** Datos que se desvían significativamente de la mayoría.
- **Valores duplicados:** Registros idénticos o casi idénticos.
- **Valores faltantes:** Campos vacíos o sin información.
- **Inconsistencias:** Datos que no cumplen con las reglas de negocio o los formatos establecidos.

Importancia de la limpieza de datos en la investigación



Limpieza y preparación de datos



Etapas típicas de la limpieza de datos

✓ Corrección de errores:

- **Eliminación:** Eliminar los registros que contengan errores irrecuperables.
 - **Imputación:** Rellenar los valores faltantes utilizando diferentes técnicas (media, mediana, moda, etc.).
 - **Corrección manual:** Revisar y corregir los errores de forma individual.
 - **Estandarización:** Unificar los formatos y unidades de medida.
- ✓ **Validación:** Verificar que los datos corregidos sean consistentes y cumplan con las reglas de negocio.



Herramientas para la limpieza de datos

Existen numerosas herramientas y software especializados para la limpieza de datos, como:

- Excel:** Para conjuntos de datos pequeños y sencillos.
- Python:** Con librerías como Pandas y NumPy, ofrece gran flexibilidad y potencia para el análisis de datos.
- R:** Similar a Python, con una fuerte comunidad y enfoque en estadística.
- SQL:** Para limpiar grandes bases de datos relacionales.
- Herramientas de ETL:** Extraen, transforman y cargan datos, incluyendo la limpieza.



Limpieza de datos en Python

Python se ha convertido en una de las herramientas más populares para el análisis de datos debido a su versatilidad y a la amplia gama de librerías disponibles. La limpieza de datos es una etapa crucial en cualquier proyecto de análisis de datos, y Python ofrece un conjunto de herramientas poderosas para llevar a cabo esta tarea.

¿Por qué Python para la limpieza de datos?

Librerías especializadas: Pandas es la librería más utilizada para manipular y limpiar datos en Python. Ofrece una amplia variedad de funciones para cargar, explorar, transformar y analizar datos.

Facilidad de uso: Python tiene una sintaxis clara y concisa, lo que facilita su aprendizaje y uso, incluso para aquellos que no son programadores expertos.

Integración con otras herramientas: Python se puede integrar fácilmente con otras herramientas de análisis de datos y machine learning.

Limpieza de datos en Python

La **limpieza de datos** es un paso necesario antes de realizar cualquier tarea relevante en análisis/ciencia de datos, desde elaborar un simple análisis gráfico, hasta implementar un modelo de aprendizaje automático.

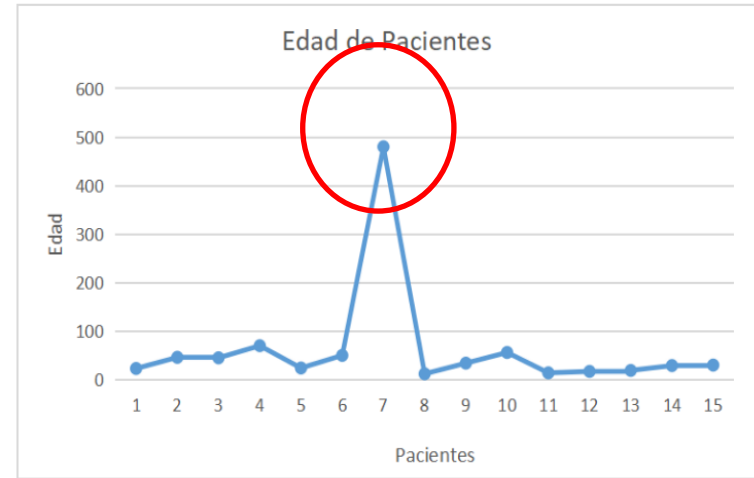
En la práctica es probablemente la tarea que más tiempo te tomará hacer. Pero no todo el proceso es incertidumbre y vicisitud. Entre las **ventajas de Python, está su facilidad** (tanto de comprensión como de ejecución) a la hora sumergirnos en este proceso..



Limpieza de datos en Python

❑ **Remove Outliers:** Por outlier nos referimos a **cualquier dato que difiere significativamente del promedio o rango observado del resto de nuestros datos.**

Aunque los outliers son mucho más fáciles de apreciar gráficamente, también pueden detectarse a primera vista en un conjunto de datos, por ejemplo, encontrar la variable *edad* de una persona en **480**. Inmediatamente nos indica que es un error de entrada de datos, probablemente la edad sea **48**.



Un ejemplo explícito de Outlier (y por consecuencia, de un mal gráfico)

❑ **Eliminar o transformar datos erróneos e irrelevantes:** Estos representan información fuera de contexto en nuestra base de datos, por ejemplo tener la variable *precio de venta* en **números negativos**.

Limpieza de datos en Python

❑ **Trabajar datos Nulos:** No se ha introducido información alguna en alguno de los elementos de la variable. Estos pueden deberse a falta de información o bien a no haber podido estimar la variable, un error de entrada, o bien hubo un fenómeno externo que impidió la recolección de datos.

Cualquiera sea la razón es importante estar consciente de esta situación, puesto que conociendo la localización y el tipo de datos que se encuentran nulos, podremos extrapolarlos, efectuar una transformación o bien, al menos conocer las limitaciones de nuestra información. Este fenómeno también puede ser conocido como **"Missing Data"**..

```
currency_name equivalent_fips_code fips geoname_id languages
0 Euro NaN AN 3041565 ca
1 Dirham NaN AE 290557 ar-AE,fa,en,hi,ur
2 Afghani NaN AF 1149361 fa-AF,ps,uz-AF,tk
3 Dollar NaN AC 3576396 en-AG
4 Dollar NaN AV 3573511 en-AI

name neighbours numeric phone population
0 Andorra ES,FR 20 376 84000
1 United Arab Emirates SA,OM 784 971 4975593
2 Afghanistan TM,CN,IR,TJ,PK,UZ 4 93 29121286
3 Antigua and Barbuda NaN 28 +1-268 86754
4 Anguilla NaN 660 +1-264 13254

postal_code_format postal_code_regex tld
0 AD### ^(?:AD)*(\d{3})$ ad
1 NaN NaN ae
```

Limpieza de Datos con Pandas

Pandas nos ofrece una variedad de opciones para resolver problemas de este tipo.

```
import pandas as pd
datos=pd.read_csv("C:Downloads\SP.csv")
datos
```

Proceso de importación de Pandas y lectura de archivo

Exploración de datos

- ✓ Ver las primeras filas: `df.head()`
- ✓ Ver las últimas filas: `df.tail()`
- ✓ Obtener información general: `df.info()`
- ✓ Ver estadísticas descriptivas: `df.describe()`

Limpieza de datos en Python

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
0	Recursos Humanos	38	10.0	NaN	NaN
1	Marketing	39	14.0	1009.0	NaN
2	Finanzas	41	15.0	1405.0	6.0
3	Proyectos	42	NaN	1450.0	7.0
4	Informatica	42	12.0	1249.0	8.0

Como se aprecia, tenemos un resumen de la plantilla de cierta empresa ficticia. Se detalla el departamento, empleados, así como los valores *promedio* de las horas de trabajo, salario y experiencia. **El problema radica no solo en la presencia de datos nulos, sino en que estos se encuentran dispersos en distintas variables o "columnas".**

Detección de datos nulos

IsNull

IsNull nos permite detectar datos nulos, simplificando este proceso independientemente de la dimensión de nuestra base de datos.

```
#agregamos la función isnull.  
datos.isnull()
```

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
0	False	False	False	True	True
1	False	False	False	False	True
2	False	False	False	False	False
3	False	False	True	False	False
4	False	False	False	False	False

Detección de datos nulos

Notnull

Una alternativa menos conocida es Notnull. Es la indicación lógica contraria, y la forma de llamar a la función es similar. Como vez, los resultados son opuestos, puedes usar la que prefieras siempre y cuando recuerdes el criterio lógico.

```
datos.notnull()
```

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
0	False	False	False	True	True
1	False	False	False	False	True
2	False	False	False	False	False
3	False	False	True	False	False
4	False	False	False	False	False

Los resultados son opuestos al `isnull()`, puedes usar la que prefieras siempre y cuando recuerdes el criterio lógico.

Como trabajar los datos nulos

Dropna

`dropna()` elimina las filas que contienen datos nulos, por tanto es muy útil para eliminar filas con varias datos nulos a la vez, ya que dichas filas no aportarían información significativa más que para unas cuantas variables.

```
datos.dropna()
```

```
datos.dropna()
```

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
2	Finanzas	41	15.0	1405.0	6.0
4	Informatica	42	12.0	1249.0	8.0

Resultado de aplicar la función `dropna()`

La **desventaja** al usar **`dropna()`** es que puede eliminar información importante, como se muestra en el ejemplo, incluso las filas que solo tenían un solo dato nulo fueron eliminadas.

Como trabajar los datos nulos

Fillna

fillna() rellena los valores nulos con un valor predeterminado. Es importante destacar que existen distintos métodos a la hora de trabajar con fillna. Esta función es muy útil cuando solo necesitas corregir un número mínimo de datos nulos.

```
datos.fillna(value=100)
```

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
0	Recursos Humanos	38	10.0	100.0	100.0
1	Marketing	39	14.0	1009.0	100.0
2	Finanzas	41	15.0	1405.0	6.0
3	Proyectos	42	100.0	1450.0	7.0
4	Informatica	42	12.0	1249.0	8.0

En el ejemplo rellenamos los datos nulos con fillna(value=100) asignándole el valor de 100 a *todos* los datos nulos.

¿Desventaja? Como habrás notado la asignación de dicho valor puede llegar a ser arbitraria. **Veamos otro método de fillna() más rápido y efectivo.**

Como trabajar los datos nulos bfill

Este método rellena los datos nulos, en base al valor de la siguiente fila.

```
datos.fillna(method="bfill")
```

	Departamento	Horas Promedio de Trabajo(semanal)	Numero de empleados	Salario Promedio Mensual	Experiencia Promedio
0	Recursos Humanos	38	10.0	1009.0	6.0
1	Marketing	39	14.0	1009.0	6.0
2	Finanzas	41	15.0	1405.0	6.0
3	Proyectos	42	12.0	1450.0	7.0
4	Informatica	42	12.0	1249.0	8.0

Ventajas: Sin duda es mucho mejor que asignar un valor arbitrario a los datos, además garantiza que los datos se mantengan dentro de un rango específico. Es especialmente útil en bases de datos de gran extensión y con poca proporción de datos nulos.

¿Y los Outliers?

Habíamos mencionado la posibilidad de detectar los outliers. Existen varias formas de hacerlo, tales como el método intercuartil, o el método de boxplot, entre otros.

Sin embargo también podemos lograr buenos resultados de forma visual.

Usando describe

```
printf(datos.describe)
```

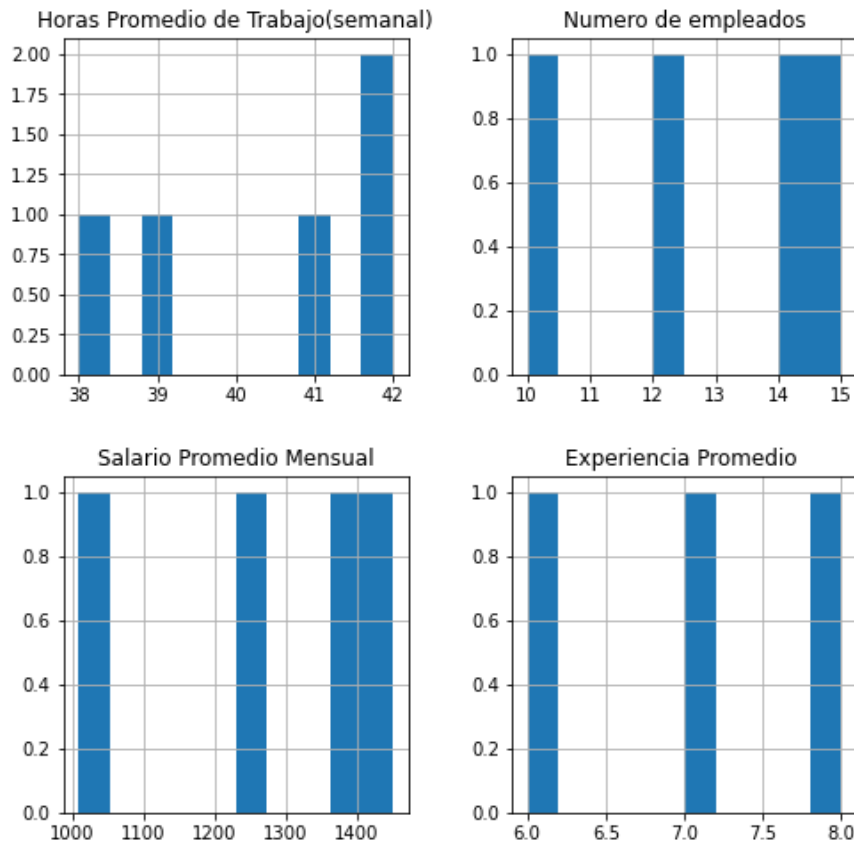
```
<bound method NDFrame.describe of
0  Recursos Humanos          38          10.0
1      Marketing            39          14.0
2      Finanzas             41          15.0
3      Proyectos            42           NaN
4      Informatica          42          12.0

   Salario Promedio Mensual  Experiencia Promedio
0                NaN          NaN
1             1009.0          NaN
2             1405.0           6.0
3             1450.0           7.0
4             1249.0           8.0 >
```

En este caso podemos observar detalladamente los diferentes valores que existen en nuestro conjunto de datos, y detectar anomalías a primera vista.

Usando un Histograma

Probemos ahora con el histograma. Simplemente empleamos `datos.hist()`



Observación por histograma. Si existe una disparidad muy notoria en la gráfica, puede deberse a un outlier.

Ambos métodos son extremadamente rápidos y simples, puesto que involucran una sola llamada a la función

Datos erróneos e irrelevantes

```
#Primero ordenaremos la información creando un DataFrame
inventario = pd.DataFrame({"Dias en Inventario": ["23", "24", "##", "14", "12", "10", "35",  
        "Precio Promedio": ["23", "24", "18", "14", "46", "44", "35", "41", "15",  
        "Tipo de Inventario": ["A", "B", "C", "B", "C", "C", "A", "A", "B", "C"],  
        "Revision Calidad": [1, 1, 0, 1, 0, 0, 1, 1, 0, 0]})
print(inventario)
```

Como resultado tenemos los siguientes:

	Dias en Inventario	Precio Promedio	Tipo de Inventario	Revision Calidad
0	23	23	A	1
1	24	24	B	1
2	##	18	C	0
3	14	14	B	1
4	12	46	C	0
5	10	44	C	0
6	35	35	A	1
7	##	41	A	1
8	40	15	B	0
9	##	9	C	0

En este caso, a diferencia del anterior, solo es una columna la que presenta problemas en los datos. Pese a esta aparente "facilidad", estos no deben tratarse como datos nulos.



Datos erróneos e irrelevantes

```
inventario.isnull()
```

	Dias en Inventario	Precio Promedio	Tipo de Inventario	Revision Calidad	Dias Inventario Corregido
0	False	False	False	False	False
1	False	False	False	False	False
2	False	False	False	False	False
3	False	False	False	False	False
4	False	False	False	False	False
5	False	False	False	False	False
6	False	False	False	False	False
7	False	False	False	False	False
8	False	False	False	False	False
9	False	False	False	False	False

Vemos pues, que isnull NO funcionará siempre y cuando exista al menos un caracter en cada elemento.

Pasemos a resolver el problema mediante el *reemplazo de datos*, por medio de una iteración reemplazando **##** por **22**, es decir, por el valor promedio de días en inventario.



Datos erróneos e irrelevantes

Vemos pues, que isnull NO funcionará siempre y cuando exista al menos un caracter en cada elemento.

Pasemos a resolver el problema mediante el *reemplazo de datos*, por medio de una iteración reemplazando **##** por **22**, es decir, por el valor promedio de días en inventario.

```
#Creamos una nueva columna con los valores ya reemplazados.  
for dias in inventario.iterrows():  
    index_value, column_values=dias  
    arreglo=int(column_values['Dias en Inventario'].replace('##','22'))  
    inventario.at[index_value,'Dias Inventario Corregido']=arreglo
```

Veamos el resultado:

inventario					
	Dias en Inventario	Precio Promedio	Tipo de Inventario	Revision Calidad	Dias Inventario Corregido
0	23	23	A	1	23.0
1	24	24	B	1	24.0
2	##	18	C	0	22.0
3	14	14	B	1	14.0
4	12	46	C	0	12.0
5	10	44	C	0	10.0
6	35	35	A	1	35.0
7	##	41	A	1	22.0
8	40	15	B	0	40.0
9	##	9	C	0	22.0



Se ha creado la nueva columna reemplazando los datos erróneos con el promedio 22. Pero debemos eliminar columna antigua

Datos erróneos e irrelevantes

```
#Eliminamos Dias en Inventario.  
inventario.drop(axis=1, columns='Dias en Inventario')
```

```
inventario.drop(axis=1, columns='Dias en Inventario')
```

	Precio Promedio	Tipo de Inventario	Revision Calidad	Dias Inventario Corregido
0	23	A	1	23.0
1	24	B	1	24.0
2	18	C	0	22.0
3	14	B	1	14.0
4	46	C	0	12.0
5	44	C	0	10.0
6	35	A	1	35.0
7	41	A	1	22.0
8	15	B	0	40.0
9	9	C	0	22.0

Hemos garantizado (o al menos mejorado), la integridad de todos los datos. Los **valores de ##** han sido reemplazados por el valor más cercano a la media; 22. **Puedes confirmar que el valor de la media es 22.4 usando `inventario['Dias Inventario Corregido'].mean()`.**

Resumen

Exploración de datos:

- Ver las primeras filas: `df.head()`
- Ver las últimas filas: `df.tail()`
- Obtener información general: `df.info()`
- Ver estadísticas descriptivas: `df.describe()`

Manejo de valores faltantes:

- Identificar valores faltantes: `df.isnull().sum()`
- Eliminar filas con valores faltantes: `df.dropna()`

Rellenar valores faltantes:

- Con un valor específico: `df.fillna(0)`
- Con la media: `df.fillna(df.mean())`
- Con la mediana: `df.fillna(df.median())`
- Con la moda: `df.fillna(df.mode())`

Detección y eliminación de duplicados:

- `df.duplicated().sum()`
- `df.drop_duplicates()`

Ejemplo práctico

Python

```
import pandas as pd
```

```
# Cargar datos
```

```
df = pd.read_csv('datos.csv')
```

```
# Explorar datos
```

```
print(df.head())
```

```
print(df.info())
```

```
# Manejar valores faltantes
```

```
df.fillna(df.mean(), inplace=True)
```

```
# Eliminar duplicados
```

```
df.drop_duplicates(inplace=True)
```

```
# Guardar datos limpios
```

```
df.to_csv('datos_limpios.csv', index=False)
```

Limpieza de datos en Python

La limpieza de datos es un **proceso iterativo** y puede requerir **varias pasadas** para obtener un conjunto de datos de alta **calidad**. Es importante **documentar** cada paso del proceso para facilitar la reproducibilidad y la colaboración.

